K.R. MANGALAM UNIVERSITY

ENSP465 | New Age Programming Languages Lab

# AI-Powered Student Performance Prediction System

**CAPSTONE PROJECT • PHASE I**

Project Proposal & System Design

Submitted by: Harman Jeet Singh | Roll No.: 2301010252
Vanshika Sharma | Roll No.: 2301010368 • Session: 2023–2027
Semester VII • Academic Year 2026–27
Faculty: Dr. Hemraj Kumawat | SOET, KRMU • Submission Date: 22 October 2026

PLANNING → DESIGN → IMPLEMENTATION

# Problem Statement & Motivation

## The problem

Student performance is shaped by multiple signals—attendance, assessment results, assignment completion and study habits. These signals are often spread across records, making it difficult to identify a struggling student early and consistently.

## Who is affected?

- Students who need timely academic support
- Faculty and mentors monitoring large classes
- Academic coordinators planning interventions

### Why it matters

Late identification can mean missed opportunities for tutoring, attendance support or study planning.

### Current limitation

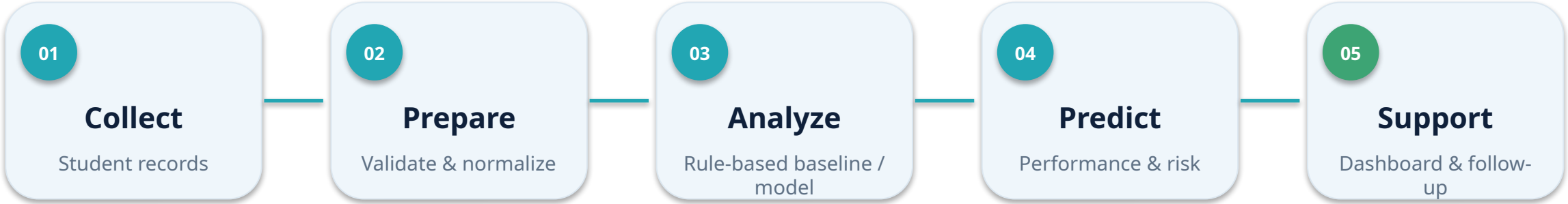
Manual review and basic grade reports describe past results but may not combine patterns into an early risk signal.

### Motivation

Provide a simple, explainable prediction and flag students who may benefit from human follow-up.

# Proposed Solution

A planned student analytics system that combines attendance, academic scores, assignment completion and study-pattern indicators to estimate a student's likely performance category. It will present a risk flag and a concise explanation of the factors contributing to the estimate, helping faculty prioritize supportive follow-up.



# Why Kotlin?

## KOTLIN

**Primary implementation language**

Chosen for reliable data handling and a clear path to a JVM-based application.

### 1. Null safety

Helps reduce null-related runtime errors when handling incomplete student records.

### 2. Data classes

Concise, structured models for Student, AttendanceRecord, Assessment and Prediction.

### 3. JVM ecosystem

Can use mature JVM libraries for CSV/JSON processing, testing and persistence.

### 4. Coroutines (if needed)

Support asynchronous tasks such as loading records or refreshing dashboard data.

# Objectives & Scope

## Measurable objectives

1. Define a student-record schema with attendance, marks, assignment status and study indicators.

2. Build a validation and preprocessing workflow for missing or inconsistent values.

3. Produce a performance category and risk flag from the selected prediction approach.

4. Display student-level results and cohort summaries in a simple interface.

5. Evaluate predictions on a sample dataset using suitable metrics.

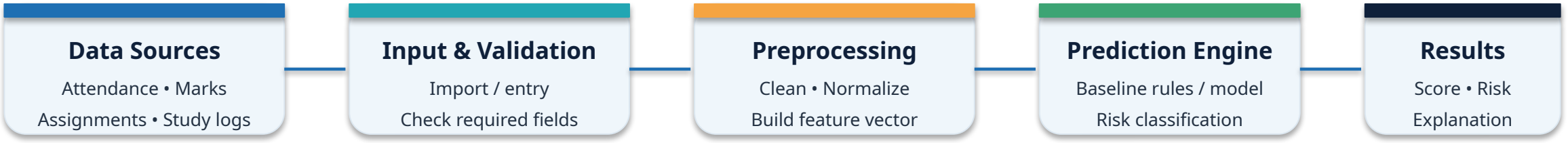
## In scope

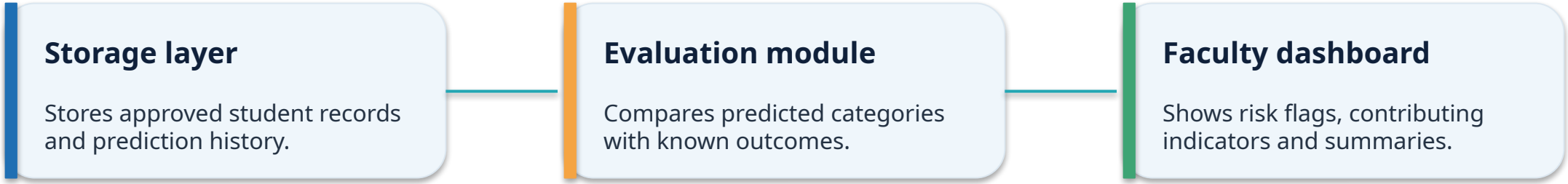
- Record import / entry
- Data validation
- Feature preparation
- Prediction and risk flag
- Results dashboard
- Basic evaluation

## Out of scope

- Automated disciplinary decisions
- Guaranteed future outcomes
- Facial / biometric tracking
- Production-scale integration
- Replacing faculty judgement

# System Architecture



# Components & Input–Process–Output

| MODULE | INPUT / PROCESS | OUTPUT |
|---|---|---|
| **Input module** | Attendance %, assessment marks, assignment completion, study frequency | Validated student record |
| **Preprocessing** | Handle missing values, normalize scales, derive attendance / trend features | Feature vector |
| **Prediction engine** | Apply baseline scoring or a selected model; assign performance band and risk flag | Predicted category + risk |
| **Dashboard / report** | Display individual summary, cohort view and key contributing factors | Actionable review list |
| **Evaluation** | Compare outputs with known results; compute accuracy / precision / recall as appropriate | Evaluation summary |

**IPO summary** INPUT: academic indicators → PROCESS: validate, engineer features, predict → OUTPUT: performance band, risk flag, supportive follow-up list

# Kotlin Design & Responsible Use

### Student data class

Student(id, attendanceRate, marks, assignmentCompletion, studyFrequency)

### Prediction data class

Prediction(studentId, performanceBand, riskLevel, contributingFactors)

### Result types

Use enums / sealed classes for risk bands and explicit success / validation errors.

## Responsible design safeguards

- Missing or unreliable data should be flagged, not silently treated as fact.
- Explain the main indicators behind a risk flag; avoid presenting a prediction as certainty.
- Limit access to student records and avoid unnecessary personal data.
- Review outcomes with faculty; provide support, not punitive automated decisions.

# Work Plan & Timeline

1

## Phase I proposal

**By 24 Oct 2026**

Problem definition, architecture, scope and references.

2

## Requirements & dataset

**Late Oct 2026**

Finalize fields, sample dataset, validation rules and prediction approach.

3

## Phase II implementation

**Nov 2026**

Build Kotlin modules, data handling, prediction logic and the initial interface.

4

## Progress demo

**Late Nov 2026**

Demonstrate the working flow and incorporate faculty feedback.

5

## Testing & refinement

**Early Dec 2026**

Run test cases, evaluate metrics, fix errors and complete documentation.

6

## Final submission

**By Dec 2026**

# References


- [1] Kotlin Documentation. “Kotlin language documentation.” https://kotlinlang.org/docs/home.html (accessed October 2026).
- [2] Kotlin Documentation. “Data classes.” https://kotlinlang.org/docs/data-classes.html (accessed October 2026).
- [3] Kotlin Documentation. “Null safety.” https://kotlinlang.org/docs/null-safety.html (accessed October 2026).
- [4] R. Asif, A. Merceron, S. A. Ali, and N. G. Haider, “Analyzing undergraduate students’ performance using educational data mining,” Computers & Education, vol. 113, pp. 177–194, 2017, doi: 10.1016/j.compedu.2017.05.007.
- [5] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning, 2nd ed. Springer, 2009. https://hastie.su.domains/ElemStatLearn/.